Panduan Membuat Doctor Agent

Multi-agent RAG chatbot di atas Medical Book — dari nol sampai jalan, langkah demi langkah, lengkap dengan penjelasan, kode, dan checkpoint.

Planner · RAG · Evaluator · Summarizer | Python · Streamlit · Claude

Panduan ini membawa kamu membangun ulang **Doctor Agent** langkah demi langkah: chatbot medis multi-agent (Planner, RAG, Evaluator, Summarizer) di atas Streamlit, yang menjawab hanya dari *Medical Book*.

Cara memakai panduan ini

- Kerjakan **berurutan**. Setiap langkah diakhiri ✅ **Checkpoint**: jangan lanjut sebelum checkpoint lulus.
- Semua kode di sini identik dengan versi di folder `production/` yang sudah teruji (20 unit test lulus, evaluasi 27 soal 100%). Kalau versi ketikanmu error, bandingkan dengan file aslinya.
- **Ketik, jangan copy-paste** kalau tujuanmu memahami. Baca dulu bagian "Penjelasan", baru ketik kodenya.
- Kotak 💡 **Kenapa begini?** berisi alasan desain, yang paling sering ditanyakan penilai.

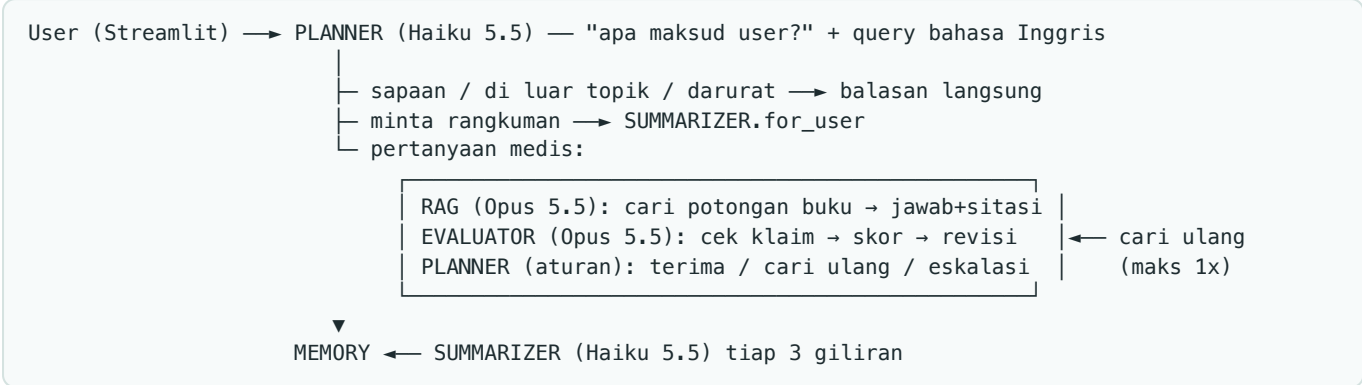
Perkiraan waktu: ±4,5–5 jam untuk langkah 0–11. Langkah 12–13 (test & evaluasi) opsional.

Daftar isi

0. Gambaran besar
1. Persiapan environment
2. Langkah 1 — `config.py`
3. Langkah 2 — `ingest.py`
4. Langkah 3 — `retriever.py`
5. Langkah 4 — `llm.py`
6. Langkah 5 — `memory.py`
7. Langkah 6 — Planner
8. Langkah 7 — RAG
9. Langkah 8 — Evaluator
10. Langkah 9 — Summarizer
11. Langkah 10 — Orchestrator
12. Langkah 11 — UI Streamlit
13. Langkah 12 — Unit test (opsional)
14. Langkah 13 — Evaluasi (opsional)
15. Troubleshooting
16. Checklist akhir

0. Gambaran besar

Apa yang dibangun



Alur data (urutan kita membangun)

#	File	Input → Output
1	`config.py`	semua angka/parameter
2	`ingest.py`	PDF → `data/chunks.json` (3.328 potongan)
3	`retriever.py`	chunks → index FAISS + BM25 → fungsi `search(query)`
4	`llm.py`	fungsi `text()` & `json()` untuk memanggil Claude + hitung biaya
5	`memory.py`	ringkasan + 3 giliran terakhir
6–9	`agents/*.py`	4 agent
10	`orchestrator.py`	menjalankan satu giliran melalui semua agent
11	`app.py`	tampilan chat

Struktur folder akhir

```
latihan_ulang/
├── .env                  ← API key (RAHASIA)
├── .env.example
├── .gitignore
├── requirements.txt
├── config.py
├── app.py
├── data/
│   ├── medical_book.pdf
│   └── chunks.json       ← dibuat oleh ingest.py
├── index/                ← dibuat oleh retriever.py
├── doctor_agent/
│   ├── __init__.py       ← file kosong
│   ├── ingest.py
│   ├── retriever.py
│   ├── llm.py
│   ├── memory.py
│   ├── orchestrator.py
│   └── agents/
│       ├── __init__.py   ← file kosong
│       ├── planner.py
│       ├── rag.py
│       ├── evaluator.py
│       └── summarizer.py
├── scripts/smoke_test.py
├── tests/test_core.py
└── eval/ (questions.json, run_eval.py)
```

1. Persiapan environment

⏱️ ±20 menit

1.1 Buat folder kerja terpisah

Jangan bekerja di `production/`, karena folder itu yang akan dikirim. Buat folder baru:

```
cd /Users/ibkindonesia/Documents/Project/Learning/DoctorAgent
mkdir latihan_ulang && cd latihan_ulang
mkdir -p data doctor_agent/agents scripts tests eval
touch doctor_agent/__init__.py doctor_agent/agents/__init__.py
cp ../data/medical_book.pdf data/
```

💡 **Kenapa ada `__init__.py` kosong?** File ini menandai folder sebagai *package* Python, sehingga kita bisa menulis `from doctor_agent.agents.planner import PlannerAgent`.

1.2 Virtual environment (Python 3.12)

Python bawaan macOS (3.9) terlalu tua untuk beberapa library. Pakai Python 3.12 dari Homebrew:

```
/opt/homebrew/opt/python@3.12/bin/python3.12 -m venv .venv
source .venv/bin/activate
python --version        # harus: Python 3.12.x
```

Setiap kali membuka Terminal baru, jalankan lagi `source .venv/bin/activate`. Tanda venv aktif: ada `(.venv)` di awal prompt.

1.3 `requirements.txt`

Buat file `requirements.txt`:

File: `requirements.txt`

```
# Python 3.11+ (developed on 3.12)
anthropic==2.7.0        # Claude API (all four agents)
streamlit==1.67.0       # web UI
pymupdf==2.1            # PDF parsing (font-aware ingestion)
fastembed==0.9.1        # local embeddings (BAAI/bge-small-en-v1.5, ONNX, CPU)
faiss-cpu==1.15.1       # vector index
rank-bm25==0.2.2        # keyword search
numpy==2.5.4
pydantic==2.14.3        # structured outputs for Planner / Evaluator
python-dotenv==1.2.4    # loads ANTHROPIC_API_KEY from .env
pandas==3.1.1
pytest==9.2.0           # offline unit tests
```

Lalu pasang:

```
pip install --upgrade pip
pip install -r requirements.txt
```

Library	Dipakai untuk

| `anthropic` | memanggil Claude (semua agent) | | `streamlit` | tampilan web chat | | `pymupdf` | membaca PDF beserta ukuran font-nya | | `fastembed` | membuat embedding secara lokal (gratis, tanpa GPU) | | `faiss-cpu` | mencari vektor yang paling mirip | | `rank-bm25` | pencarian kata kunci | | `pydantic` | mendefinisikan format JSON output Planner/Evaluator | | `python-dotenv` | membaca API key dari `.env` |

1.4 API key: `.env`, `.env.example`, `.gitignore`

`.env` (isi dengan key kamu sendiri, **jangan** di-commit):

```
ANTHROPIC_API_KEY=sk-ant-api03-xxxxxxxx
```

Cara cepat: salin dari `production`:

```
cp ../production/.env .env
```

`.env.example` (contoh tanpa rahasia, untuk penilai):

File: `.env.example`

```
# Salin file ini menjadi .env lalu isi dengan API key Anda sendiri
# Buat key di https://platform.claude.com -> API Keys
ANTHROPIC_API_KEY=sk-ant-isi-key-anda-di-sini
```

`.gitignore`:

File: `.gitignore`

```
# Rahasia - jangan pernah di-commit
.env

# Environment Python
.venv/
__pycache__/
*.pyc

# File hasil generate (bisa dibuat ulang)
index/
.models/
.pytest_cache/
.DS_Store
dist/
```

✅ Checkpoint 1

```
python -c "import anthropic, streamlit, pymupdf, fastembed, faiss, rank_bm25; print('semua library OK')"
```

Langkah 1 — `config.py`: semua parameter di satu tempat

⏱ ±10 menit · file: `config.py` (di root folder, bukan di `doctor_agent/`)

Penjelasan

Semua angka yang bisa di-*tuning* dikumpulkan di satu file. Keuntungannya:

- Ganti model, ukuran chunk, atau ambang confidence cukup di satu tempat.
- Saat presentasi, slide "Model architecture and parameters" tinggal membaca file ini.

Bagian-bagiannya:

Bagian	Isi	Catatan

| `load_dotenv(...)` | membaca `.env` → `ANTHROPIC_API_KEY` masuk ke environment | SDK Anthropic otomatis membaca env var ini | | paths | lokasi PDF, chunks, index | `ROOT` = folder tempat `config.py` berada | | ingestion | `FIRST_CONTENT_PAGE = 13` | 13 halaman pertama PDF adalah sampul/daftar isi | | retrieval | `TOP_K = 5`, `CANDIDATES = 30`, `RRF_K = 60` | dijelaskan di Langkah 3 | | LLMs | `MODELS`, `EFFORT`, `MAX_TOKENS` | model per agent ("kombinasi hemat") | | `PRICES` | harga per 1 juta token | untuk menghitung biaya di UI | | policy | ambang 0,70 / 0,45, maks 2 percobaan | dipakai Planner (Langkah 6) | | `CONFIDENCE_WEIGHTS` | 0,5 / 0,3 / 0,2 | dipakai Evaluator (Langkah 8) |

💡 **Kenapa `effort`?** Model Claude terbaru selalu "berpikir" sebelum menjawab. `effort` mengatur seberapa dalam: `low` untuk tugas ringan (lebih cepat dan murah), `medium` untuk Evaluator yang harus teliti.

Kode

File: `config.py`

```python
"""Central configuration — every tunable parameter of the system lives here."""
from pathlib import Path

from dotenv import load_dotenv

ROOT = Path(__file__).resolve().parent
load_dotenv(ROOT / ".env")  # puts ANTHROPIC_API_KEY into the environment

# ---------------------------------------------------------------- paths
DATA_DIR = ROOT / "data"
PDF_PATH = DATA_DIR / "medical_book.pdf"
CHUNKS_PATH = DATA_DIR / "chunks.json"
INDEX_DIR = ROOT / "index"

# ---------------------------------------------------------------- ingestion
FIRST_CONTENT_PAGE = 13  # 0-based; pages before this are front matter
CHUNK_CHARS = 1200       # target chunk size (~300 tokens)
OVERLAP_CHARS = 200      # overlap between consecutive sub-chunks of one section

# ---------------------------------------------------------------- retrieval
EMBED_MODEL = "BAAI/bge-small-en-v1.5"  # 384-dim, runs locally on CPU via ONNX (free)
EMBED_CACHE = ROOT / ".models"           # downloaded embedding model lives here
TOP_K = 5                # chunks handed to the RAG agent
CANDIDATES = 30          # candidates taken from each retriever before fusion
```

```
RRF_K = 60               # Reciprocal Rank Fusion constant

# ---------------------------------------------------------------- LLMs
# "Cost-saving mix": cheap fast model for routing/summarising,
# strongest model where medical accuracy matters (answering + fact-checking).
MODELS = {
    "planner": "claude-haiku-5-5",
    "rag": "claude-opus-5-5",
    "evaluator": "claude-opus-5-5",
    "summarizer": "claude-haiku-5-5",
}
EFFORT = {  # thinking depth per agent: low | medium | high
    "planner": "low",
    "rag": "low",
    "evaluator": "medium",
    "summarizer": "low",
}
MAX_TOKENS = {"planner": 1500, "rag": 4000, "evaluator": 6000, "summarizer": 1500}

# USD per 1M tokens (input, output) - used for the live cost counter
PRICES = {
    "claude-opus-5-5": (4.00, 20.00),
    "claude-sonnet-5-5": (2.00, 10.00),
    "claude-haiku-5-5": (0.10, 0.50),
}

# ---------------------------------------------------------------- planner policy
ACCEPT_THRESHOLD = 0.70    # confidence >= this -> show answer
ESCALATE_THRESHOLD = 0.45  # still below this after re-query -> escalate
MAX_ATTEMPTS = 2           # 1 answer + at most 1 re-query
SUMMARY_EVERY = 3          # Summarizer runs every N turns
RECENT_TURNS = 3           # raw turns kept next to the running summary

# confidence = weighted mix of the Evaluator's scores
CONFIDENCE_WEIGHTS = {"faithfulness": 0.5, "completeness": 0.3, "relevance": 0.2}
```

✅ Checkpoint

```
python -c "import config, os; print(config.PDF_PATH.exists(),
bool(os.environ.get('ANTHROPIC_API_KEY')))"
# harus: True True
```

Langkah 2 — `ingest.py`: PDF menjadi chunk

⏱️ ±30 menit · file: `doctor_agent/ingest.py` · (kamu sudah pernah membuat ini di `latihan/ingest_saya.py`)

Konsep: chunking berbasis struktur

RAG butuh dokumen yang dipotong kecil-kecil (*chunk*), karena kita hanya mengirim 5 potongan paling relevan ke LLM, bukan 637 halaman.

Cara naif: potong setiap 1.000 karakter. Masalahnya, satu potongan bisa berisi akhir artikel "Acne" dan awal artikel "Acoustic neuroma", sehingga LLM bingung.

Cara kita: buku ensiklopedia punya **pola font yang konsisten**:

Ukuran font	Artinya	Contoh
15 pt, bold	judul entri (penyakit)	**ACNE**
11 pt, bold	judul section	**Causes and symptoms**
10 pt	isi	Acne vulgaris affects...
12,5 pt "KEY TERMS"	kotak istilah	
12 pt	rujukan silang	"Abrasions see Wounds"
≤ 9 pt	daftar pustaka/caption	dibuang

Jadi kita baca setiap blok teks beserta font-nya, lalu memotong **mengikuti struktur buku**. Hasilnya, setiap chunk tahu: entri apa, section apa, halaman berapa. Info inilah yang nanti menjadi sitasi.

Penjelasan per fungsi

Fungsi	Tugas
`Section` (dataclass)	wadah sementara: entry, section, halaman, daftar baris teks
`_block_kind(spans)`	melihat font span pertama sebuah blok → mengembalikan jenisnya (`entry`, `section`, `body`, ...)
`_page_blocks(page)`	mengambil semua blok di satu halaman, membuang header/footer, **mengurutkan 2 kolom** (kolom kiri dulu, baru kanan), dan memisahkan judul yang tergabung dengan paragraf
`_join_lines(lines)`	menggabungkan baris, sekaligus memperbaiki pemenggalan kata ("vacu-" + "um" → "vacuum")
`_book_page(page)`	mencari nomor halaman cetak (angka 10,1 pt), lebih enak dikutip daripada nomor halaman PDF
`parse_sections()`	*state machine* utama: berjalan halaman demi halaman, menyimpan entri dan section aktif, membuang bibliografi (`mode = "skip"` setelah "Resources"), mengumpulkan rujukan silang menjadi `aliases`
`split_text()`	section yang lebih panjang dari 1.200 karakter dipecah di batas kalimat, dengan overlap 200 karakter
`build_chunks()`	menggabungkan semuanya menjadi daftar dict `{id, entry, section, pdf_page, book_page, text}`
`main()`	menyimpan ke `data/chunks.json`

💡 **Kenapa ada overlap 200 karakter?** Supaya kalimat yang berada di perbatasan dua potongan tidak kehilangan konteks.

💡 **Apa itu `aliases` ?** Buku punya rujukan seperti "*Abdominal hernia see Hernia*". Kita simpan sebagai kamus; kalau user menyebut "abdominal hernia", query ditambah "Hernia".

Kode

File: `doctor_agent/ingest.py`

```
"""Step 1 – Ingestion: turn the Medical Book PDF into structure-aware chunks.

The book (Gale Encyclopedia of Medicine, vol. 1, A-B) has a very regular layout:
    ENTRY TITLE (15pt bold)  ->  Section heading (11pt bold)  ->  body text
so instead of cutting the text blindly every N characters we read the font
size/weight of every text block and cut along the book's own structure.
Every chunk therefore knows which entry + section + page it came from, which
we later show to the user as a citation.

Run:  python -m doctor_agent.ingest
"""
from __future__ import annotations

import json
import re
import sys
from dataclasses import dataclass, field
from pathlib import Path

import pymupdf

from config import CHUNK_CHARS, CHUNKS_PATH, FIRST_CONTENT_PAGE, OVERLAP_CHARS, PDF_PATH


@dataclass
class Section:
    entry: str
    section: str
    pdf_page: int
    book_page: str
    lines: list[str] = field(default_factory=list)


def _block_kind(spans: list[dict]) -> str:
    """Classify a text block by the font of its first span."""
    s0 = spans[0]
    size, font, text = round(s0["size"], 1), s0["font"], s0["text"].strip()
    if size == 15.0 and "Bold" in font:
        return "entry"
    if size == 12.5 and text.upper().startswith("KEY TERMS"):
        return "keyterms_header"
    if size == 12.0:
        return "crossref"  # "Abrasions see Wounds"
    if size == 11.0 and "Bold" in font:
        return "section"
    if size == 10.0 and font == "Optima-Bold" and text == "Resources":
        return "resources"
    if size == 10.0 and font == "Optima-Bold":
        return "keyterm"  # "Term - definition" inside the KEY TERMS box
    if size <= 9.0:
        return "small"  # bibliography / captions
    return "body"


def _page_blocks(page: pymupdf.Page):
    """Yield (kind, lines) for content blocks in reading order (2 columns)."""
    w, h = page.rect.width, page.rect.height
    blocks = []
    for b in page.get_text("dict")["blocks"]:
        if "lines" not in b:
            continue
        x0, y0, x1, _ = b["bbox"]
        if y0 > h * 0.91 or x1 < 60 or x0 > w - 60:  # footer / margin header
```

```
            continue
        spans = [s for l in b["lines"] for s in l["spans"] if s["text"].strip()]
        if not spans:
            continue
        lines = ["".join(s["text"] for s in l["spans"]) for l in b["lines"]]
        col = 0 if x0 < w / 2 - 10 else 1
        kind = _block_kind(spans)
        if kind in ("entry", "section"):
            # A heading and the paragraph after it sometimes share one block:
            # keep only the heading-sized lines as heading, re-classify the rest.
            head = round(spans[0]["size"], 1)
            sizes = [
                next((round(s["size"], 1) for s in l["spans"] if s["text"].strip()), head)
                for l in b["lines"]
            ]
            n = next((i for i, sz in enumerate(sizes) if sz != head), len(lines))
            if n < len(lines):
                rest = [s for l in b["lines"][n:] for s in l["spans"] if s["text"].strip()]
                blocks.append((col, y0, kind, lines[:n]))
                blocks.append((col, y0 + 0.1, _block_kind(rest), lines[n:]))
                continue
        blocks.append((col, y0, kind, lines))
    blocks.sort(key=lambda t: (t[0], t[1]))  # left column first, then top-down
    for _, _, kind, lines in blocks:
        yield kind, lines


def _join_lines(lines: list[str]) -> str:
    """Join PDF lines, repairing end-of-line hyphenation ("vacu-" + "um")."""
    out = ""
    for ln in lines:
        ln = re.sub(r"\s+", " ", ln).strip()
        if not ln:
            continue
        if out.endswith("-") and ln[:1].islower():
            out = out[:-1] + ln
        else:
            out = f"{out} {ln}" if out else ln
    return out


def _book_page(page: pymupdf.Page) -> str:
    """The printed page number (10.1pt digits) - nicer to cite than the PDF index."""
    for b in page.get_text("dict")["blocks"]:
        for l in b.get("lines", []):
            for s in l["spans"]:
                if round(s["size"], 1) == 10.1 and s["text"].strip().isdigit():
                    return s["text"].strip()
    return ""


def parse_sections(pdf_path: Path = PDF_PATH) -> tuple[list[Section], dict[str, str]]:
    doc = pymupdf.open(pdf_path)
    sections: list[Section] = []
    aliases: dict[str, str] = {}
    cur: Section | None = None
    keyterms: Section | None = None
    entry, mode = "", "body"  # mode: body (keep text) | skip (bibliography)
    pending_title: list[str] = []
    crossref_buf: list[str] = []

    def start(sec_name: str, pno: int, bp: str) -> Section:
        s = Section(entry, sec_name, pno + 1, bp)
        sections.append(s)
        return s

    for pno in range(FIRST_CONTENT_PAGE, doc.page_count):
        page = doc[pno]
```

```
        bp = _book_page(page)
        for kind, lines in _page_blocks(page):
            text = _join_lines(lines)
            if kind != "entry" and pending_title:
                # entry titles can wrap over several blocks — close the title now
                entry = " ".join(pending_title)
                pending_title = []
                cur, keyterms, mode = start("Overview", pno, bp), None, "body"
            if kind == "crossref":
                crossref_buf.append(text)
                continue
            if kind == "entry":
                pending_title.append(text)
                continue
            if kind == "section":
                cur, mode = start(text, pno, bp), "body"
            elif kind == "resources":
                mode = "skip"
            elif kind == "keyterms_header":
                keyterms = keyterms or start("Key terms", pno, bp)
            elif kind == "keyterm":
                if entry:
                    keyterms = keyterms or start("Key terms", pno, bp)
                    keyterms.lines.append(text)
            elif kind == "body" and mode == "body" and cur is not None:
                cur.lines.append(text)

    # Cross references: "Abrasions see Wounds Abruptio placentae see Placental abruption"
    joined = " ".join(crossref_buf)
    for m in re.finditer(r"(.+?)\s+see\s+(.+?)(?=\s+[A-Z][\w,''\- ]*?\s+see\s+|$)", joined):
        aliases[m.group(1).strip()] = m.group(2).strip()
    return [s for s in sections if s.lines], aliases


def split_text(text: str, size: int = CHUNK_CHARS, overlap: int = OVERLAP_CHARS) -> list[str]:
    """Split long sections on sentence boundaries, with a small word-aligned overlap."""
    if len(text) <= size:
        return [text]
    sents = []
    for s in re.split(r"(?<=[.!?])\s+|\s(?=• )", text):  # sentences and bullet items
        while len(s) > size:  # very long run-on: hard split on a word boundary
            cut = s.rfind(" ", 0, size)
            cut = cut if cut > 0 else size
            sents.append(s[:cut])
            s = s[cut:].strip()
        sents.append(s)
    chunks, buf = [], ""
    for s in sents:
        if buf and len(buf) + len(s) > size:
            chunks.append(buf.strip())
            buf = buf[-overlap:].split(" ", 1)[-1] + " "  # overlap on word boundary
        buf += s + " "
    if buf.strip():
        chunks.append(buf.strip())
    return chunks


def build_chunks(pdf_path: Path = PDF_PATH) -> tuple[list[dict], dict[str, str]]:
    sections, aliases = parse_sections(pdf_path)
    chunks = []
    for s in sections:
        body = _join_lines(s.lines) if s.section != "Key terms" else "\n".join(s.lines)
        for part in split_text(body):
            chunks.append({
                "id": f"{len(chunks):05d}",
                "entry": s.entry,
                "section": s.section,
                "pdf_page": s.pdf_page,
```

```
                "book_page": s.book_page,
                "text": part,
            })
    return chunks, aliases


def main() -> None:
    chunks, aliases = build_chunks()
    CHUNKS_PATH.write_text(json.dumps({"chunks": chunks, "aliases": aliases}, indent=1,
ensure_ascii=False))
    entries = {c["entry"] for c in chunks}
    print(f"{len(chunks)} chunks, {len(entries)} entries, {len(aliases)} aliases -> {CHUNKS_PATH}",
file=sys.stderr)


if __name__ == "__main__":
    main()
```

✅ Checkpoint

```
python -m doctor_agent.ingest
# harus: 3328 chunks, 282 entries, 268 aliases -> .../data/chunks.json
```

Catatan: jalankan dengan `python -m doctor_agent.ingest` dari **root folder** (bukan `python doctor_agent/ingest.py`), supaya `import config` berhasil.

Langkah 3 — `retriever.py` : hybrid search

⏱ ±50 menit · file: `doctor_agent/retriever.py` · **bagian paling konseptual, pahami betul**

Konsep 1 — Embedding & dense search

Embedding mengubah teks menjadi vektor (di sini 384 angka). Teks yang **maknanya mirip** punya vektor yang **arahnya mirip**. Kemiripan diukur dengan *cosine similarity* (1 = sama persis, 0 = tidak berhubungan).

- Model: `BAAI/bge-small-en-v1.5`, kecil (±130 MB), berjalan di CPU, gratis.
- `passage_embed()` untuk dokumen, `query_embed()` untuk pertanyaan (model BGE menambahkan instruksi khusus untuk query).
- Kita embed `"Entry – Section: teks"` supaya vektornya tahu topiknya.

FAISS menyimpan semua vektor dan mencari yang paling mirip. `IndexFlatIP` = pencarian *exact* dengan *inner product*. Karena vektor sudah dinormalisasi (`normalize_L2`), inner product = cosine similarity.

💡 **Kenapa FlatIP, bukan index aproksimasi (IVF/HNSW)?** 3.328 vektor itu kecil; pencarian exact < 1 ms. Index aproksimasi baru perlu untuk jutaan dokumen.

Konsep 2 — BM25 (keyword search)

BM25 menilai dokumen berdasarkan **kata yang sama persis** dengan query, dengan bobot lebih tinggi untuk kata yang langka (misalnya "acetaminophen" lebih bermakna daripada "disease").

- `tokenize()`: huruf kecil, ambil kata alfanumerik, buang *stopwords* ("the", "is", ...).

Konsep 3 — Reciprocal Rank Fusion (RRF)

Dense bagus untuk sinonim ("high blood pressure" ≈ "hypertension"); BM25 bagus untuk istilah persis. Kita jalankan keduanya lalu menggabungkan **peringkatnya** (bukan skornya, karena skala skornya berbeda):

```
skor_RRF(dokumen) = Σ  1 / (k + peringkat)      dengan k = 60
```

Contoh: dokumen A peringkat 1 di dense dan 3 di BM25 → 1/61 + 1/63 = 0,0323. Dokumen B peringkat 2 di dense saja → 1/62 = 0,0161. A menang karena bagus di keduanya.

Tambahan kecil: kalau query menyebut **judul entri** persis ("acne"), chunk dari entri itu diberi bonus.

Penjelasan per bagian kode

Bagian	Tugas
`Hit`	hasil pencarian: chunk + skor RRF + `dense_sim` (cosine, dipakai sebagai sinyal kepercayaan)
`build()`	embed semua chunk → simpan index ke `index/chunks.faiss` (±2 menit, sekali saja)
`load()`	baca index yang sudah ada; kalau belum ada, otomatis `build()`
`_expand()`	tambahkan target alias ke query. **Pakai regex `\b...\b` (kata utuh)!**
`search()`	dense top-30 + BM25 top-30 → RRF → bonus judul → ambil top-5
parameter `mode`	`hybrid` / `dense` / `sparse`, untuk eksperimen ablation di evaluasi

⚠️ **Bug yang pernah terjadi (bagus untuk diceritakan saat interview):** versi awal `_expand` memakai `if alias in q` (pencocokan substring). Alias **"ALS"** ikut cocok di dalam kata "p**als**y", sehingga query "Bell's palsy" tercemar artikel ALS. Solusinya regex kata utuh `\b...\b`.

Kode

File: `doctor_agent/retriever.py`

```
"""Step 2 — Hybrid retrieval: dense vectors (FAISS) + keyword search (BM25).

Why hybrid?  Dense embeddings understand meaning ("high blood pressure" ~
"hypertension"), but medical text is full of exact technical terms and drug
names where plain keyword matching is more reliable.  We run both and merge
their rankings with Reciprocal Rank Fusion (RRF):  score = sum 1 / (k + rank).

Build the index once:  python -m doctor_agent.retriever
"""
from __future__ import annotations

import json
import re
import sys
from dataclasses import dataclass

import faiss
import numpy as np
from rank_bm25 import BM25Okapi

from config import CANDIDATES, CHUNKS_PATH, EMBED_CACHE, EMBED_MODEL, INDEX_DIR, RRF_K, TOP_K

_STOP = set(
    "a an and are as at be by can do does for from has have how i in is it its of on or "
    "that the this to was what when where which who why will with you your about my me".split()
)


def tokenize(text: str) -> list[str]:
    return [t for t in re.findall(r"[a-z0-9]+", text.lower()) if t not in _STOP]


def chunk_to_passage(c: dict) -> str:
    """Text that gets embedded: prefix with entry + section so the vector knows the topic."""
    return f"{c['entry']} — {c['section']}: {c['text']}"


@dataclass
class Hit:
    chunk: dict
    score: float      # fused RRF score (used for ranking)
    dense_sim: float  # cosine similarity with the query (0..1, used as a signal)


class HybridRetriever:
    def __init__(self, chunks: list[dict], aliases: dict[str, str], index: faiss.Index, embedder):
        self.chunks = chunks
        self.aliases = {k.lower(): v for k, v in aliases.items()}
        self.entries = {c["entry"].lower(): c["entry"] for c in chunks}
        self.index = index
        self.embedder = embedder
        self.bm25 = BM25Okapi([tokenize(chunk_to_passage(c)) for c in chunks])

    # ---------------------------------------------------------- build / load
    @staticmethod
```

```
    def _embedder():
        from fastembed import TextEmbedding  # imported lazily: slow-ish import
        return TextEmbedding(model_name=EMBED_MODEL, cache_dir=str(EMBED_CACHE))

    @classmethod
    def build(cls) -> "HybridRetriever":
        data = json.loads(CHUNKS_PATH.read_text())
        chunks, aliases = data["chunks"], data["aliases"]
        embedder = cls._embedder()
        vecs = np.array(list(embedder.passage_embed([chunk_to_passage(c) for c in chunks])),
dtype="float32")
        faiss.normalize_L2(vecs)                # cosine similarity == inner product
        index = faiss.IndexFlatIP(vecs.shape[1])  # exact search; 3.3k vectors is tiny
        index.add(vecs)
        INDEX_DIR.mkdir(exist_ok=True)
        faiss.write_index(index, str(INDEX_DIR / "chunks.faiss"))
        return cls(chunks, aliases, index, embedder)

    @classmethod
    def load(cls) -> "HybridRetriever":
        path = INDEX_DIR / "chunks.faiss"
        if not path.exists():
            return cls.build()
        data = json.loads(CHUNKS_PATH.read_text())
        return cls(data["chunks"], data["aliases"], faiss.read_index(str(path)), cls._embedder())

    # ------------------------------------------------------------ search
    def _expand(self, query: str) -> str:
        """Resolve the book's cross references ("Abdominal hernia see Hernia")."""
        q = query.lower()
        # whole-word match only: a plain substring test made "ALS" fire inside "palsy"
        extra = [target for alias, target in self.aliases.items() if
re.search(rf"\b{re.escape(alias)}\b", q)]
        return query + (" " + " ".join(extra) if extra else "")

    def search(self, query: str, k: int = TOP_K, mode: str = "hybrid") -> list[Hit]:
        """mode: 'hybrid' (default), 'dense' or 'sparse' - the last two exist for the ablation
study."""
        query = self._expand(query)

        # dense ranking
        qv = np.array(list(self.embedder.query_embed(query)), dtype="float32")
        faiss.normalize_L2(qv)
        sims, ids = self.index.search(qv, CANDIDATES)
        dense = {int(i): float(s) for i, s in zip(ids[0], sims[0]) if i >= 0}

        # keyword ranking
        bm = self.bm25.get_scores(tokenize(query))
        sparse = [int(i) for i in np.argsort(bm)[::-1][:CANDIDATES] if bm[i] > 0]

        # reciprocal rank fusion
        fused: dict[int, float] = {}
        if mode in ("hybrid", "dense"):
            for rank, i in enumerate(dense):
                fused[i] = fused.get(i, 0) + 1 / (RRF_K + rank + 1)
        if mode in ("hybrid", "sparse"):
            for rank, i in enumerate(sparse):
                fused[i] = fused.get(i, 0) + 1 / (RRF_K + rank + 1)

        # small boost when the query names an entry title exactly ("acne", "bursitis")
        ql = query.lower()
        for i in (fused if mode == "hybrid" else []):
            entry = self.chunks[i]["entry"].lower()
            if len(entry) > 3 and re.search(rf"\b{re.escape(entry)}\b", ql):
                fused[i] += 1 / RRF_K

        best = sorted(fused, key=fused.get, reverse=True)[:k]
        missing = [i for i in best if i not in dense]
```

```
        if missing:  # cosine sim for BM25-only hits, so every hit has a comparable signal
            vecs = np.vstack([self.index.reconstruct(i) for i in missing])
            for i, s in zip(missing, (vecs @ qv[0]).tolist()):
                dense[i] = s
        return [Hit(self.chunks[i], fused[i], dense[i]) for i in best]


def main() -> None:
    r = HybridRetriever.build()
    print(f"indexed {r.index.ntotal} chunks -> {INDEX_DIR}", file=sys.stderr)
    for q in ["What causes acne?", "treatment for bronchitis", "abdominal hernia"]:
        hits = r.search(q, 3)
        print(q, "->", [(h.chunk["entry"], h.chunk["section"], round(h.dense_sim, 2)) for h in hits])


if __name__ == "__main__":
    main()
```

✅ Checkpoint

```
python -m doctor_agent.retriever
```

Pertama kali akan mengunduh model embedding lalu meng-embed 3.328 chunk (±2 menit). Output yang diharapkan:

```
indexed 3328 chunks -> .../index
What causes acne? -> [('Acne', 'Causes and symptoms', 0.88), ...]
treatment for bronchitis -> [('Bronchitis', 'Treatment', 0.84), ...]
```

Langkah 4 — `llm.py` : pembungkus Claude

⏱️ ±20 menit · file: `doctor_agent/llm.py`

Penjelasan

Keempat agent memanggil Claude dengan cara yang sama, jadi kita buat satu pembungkus:

- `text(agent, system, messages)` → jawaban teks biasa (dipakai RAG & Summarizer)
- `json(agent, system, messages, schema)` → jawaban **JSON terstruktur** yang langsung divalidasi menjadi objek Pydantic (dipakai Planner & Evaluator)

Setiap panggilan mengembalikan `Usage` (agent, model, token input/output, detik), dan `cost` dihitung dari `PRICES` di config. Inilah sumber angka biaya dan latensi di UI dan evaluasi.

Detail penting:

Bagian	Penjelasan
`output_config={"effort": ...}`	kedalaman berpikir model per agent
`output_config["format"]`	*structured outputs*: Claude **dijamin** mengembalikan JSON sesuai schema
`_strict_schema()`	schema dari Pydantic diubah supaya semua field wajib (`required`) dan tidak boleh ada field lain (`additionalProperties: false`), karena structured outputs mensyaratkan itu
`FALLBACK_BETA` + `fallbacks="default"`	khusus Opus 4.5: kalau filter keamanan keliru menolak pertanyaan medis yang sah, API otomatis mencoba model cadangan dalam panggilan yang sama
`stop_reason == "refusal"`	kalau tetap ditolak, kita lempar `LLMRefusal`, lalu orchestrator mengubahnya jadi eskalasi
`max_retries=3`	SDK otomatis mengulang kalau kena rate limit / error server

💡 **Kenapa structured output, bukan "tolong jawab dalam JSON"?** Tanpa schema, LLM kadang menambahkan teks di luar JSON atau lupa field, sehingga kode crash. Dengan structured output, formatnya dijamin.

Kode

File: `doctor_agent/llm.py`

```
"""Thin wrapper around the Anthropic SDK shared by all agents.

It gives every agent two calls – `text()` and `json()` – and records token
usage + cost + latency of every call, so the UI and the evaluation can show
exactly what each agent costs.
"""
from __future__ import annotations

import json
import time
from dataclasses import dataclass
from typing import TypeVar

import anthropic
from pydantic import BaseModel

from config import EFFORT, MAX_TOKENS, MODELS, PRICES
```

```python
T = TypeVar("T", bound=BaseModel)

# Opus 5.5 supports server-side refusal fallbacks: if a safety classifier
# wrongly declines a (legitimate) medical question, the API retries it on a
# fallback model inside the same call instead of failing.
FALLBACK_BETA = "server-side-fallback-2026-07-01"
FALLBACK_MODELS = {"claude-opus-5-5"}


class LLMRefusal(Exception):
    """The model declined to answer (stop_reason == 'refusal')."""


@dataclass
class Usage:
    agent: str
    model: str
    input_tokens: int
    output_tokens: int
    seconds: float

    @property
    def cost(self) -> float:
        p_in, p_out = PRICES.get(self.model, (0.0, 0.0))
        return (self.input_tokens * p_in + self.output_tokens * p_out) / 1_000_000


def _strict_schema(model: type[BaseModel]) -> dict:
    """Pydantic schema -> JSON schema accepted by structured outputs
    (every field required, no extra keys)."""
    schema = model.model_json_schema()

    def fix(node):
        if isinstance(node, dict):
            node.pop("title", None)
            node.pop("default", None)
            if node.get("type") == "object" and "properties" in node:
                node["additionalProperties"] = False
                node["required"] = list(node["properties"])
            for v in node.values():
                fix(v)
        elif isinstance(node, list):
            for v in node:
                fix(v)

    fix(schema)
    return schema


class LLM:
    def __init__(self, client: anthropic.Anthropic | None = None):
        self.client = client or anthropic.Anthropic(max_retries=3)

    def _call(self, agent: str, system: str, messages: list[dict], fmt: dict | None = None):
        model = MODELS[agent]
        output_config: dict = {"effort": EFFORT[agent]}
        if fmt:
            output_config["format"] = fmt
        kwargs = dict(
            model=model,
            max_tokens=MAX_TOKENS[agent],
            system=system,
            messages=messages,
            output_config=output_config,
        )
        t0 = time.perf_counter()
        if model in FALLBACK_MODELS:
```

```
            resp = self.client.beta.messages.create(betas=[FALLBACK_BETA], fallbacks="default",
**kwargs)
        else:
            resp = self.client.messages.create(**kwargs)
        usage = Usage(agent, model, resp.usage.input_tokens, resp.usage.output_tokens,
time.perf_counter() - t0)
        if resp.stop_reason == "refusal":
            raise LLMRefusal(getattr(resp.stop_details, "explanation", "") or "declined")
        text = "".join(b.text for b in resp.content if b.type == "text").strip()
        return text, usage

    def text(self, agent: str, system: str, messages: list[dict]) -> tuple[str, Usage]:
        return self._call(agent, system, messages)

    def json(self, agent: str, system: str, messages: list[dict], schema: type[T]) -> tuple[T, Usage]:
        fmt = {"type": "json_schema", "schema": _strict_schema(schema)}
        text, usage = self._call(agent, system, messages, fmt)
        return schema.model_validate(json.loads(text)), usage
```

✅ Checkpoint (memanggil API, biaya < $0,01)

```
python -c "
from doctor_agent.llm import LLM
text, u = LLM().text('planner', 'Answer in one sentence.', [{'role':'user','content':'What is
asthma?'}])
print(text); print(u.model, u.input_tokens, u.output_tokens, f'\${u.cost:.5f}')
"
```

Langkah 5 — `memory.py`: memori percakapan

⏱ ±10 menit · file: `doctor_agent/memory.py`

Penjelasan

LLM tidak punya ingatan; setiap panggilan harus diberi konteks. Mengirim seluruh riwayat chat itu boros dan lama-lama terlalu panjang. Solusinya:

- `summary`: ringkasan berjalan, ditulis Summarizer setiap 3 giliran
- `turns`: semua giliran; yang dikirim ke prompt hanya **3 terakhir** (`RECENT_TURNS`)
- `turns_since_summary`: penghitung kapan Summarizer harus jalan lagi
- `context()`: menghasilkan teks "Conversation summary so far: ... Most recent turns: ..." untuk disisipkan ke prompt Planner dan RAG

Dengan ini, follow-up seperti "*bagaimana cara mengobatinya?*" tetap dipahami.

Kode

File: `doctor_agent/memory.py`

```python
"""Conversation memory: a running summary (written by the Summarizer agent)
plus the last few raw turns.  This keeps prompts short no matter how long
the chat gets, while still letting the bot resolve follow-ups like
"and how is it treated?"."""
from __future__ import annotations

from dataclasses import dataclass, field

from config import RECENT_TURNS


@dataclass
class Turn:
    user: str
    assistant: str


@dataclass
class Memory:
    summary: str = ""
    turns: list[Turn] = field(default_factory=list)
    turns_since_summary: int = 0

    def add(self, user: str, assistant: str) -> None:
        self.turns.append(Turn(user, assistant))
        self.turns_since_summary += 1

    def recent(self, n: int = RECENT_TURNS) -> list[Turn]:
        return self.turns[-n:]

    def unsummarized(self) -> list[Turn]:
        return self.turns[-self.turns_since_summary:] if self.turns_since_summary else []

    def context(self, n: int = RECENT_TURNS) -> str:
        """Plain-text block injected into agent prompts."""
        parts = []
        if self.summary:
            parts.append(f"Conversation summary so far:\n{self.summary}")
```

```
        recent = self.recent(n)
        if recent:
            lines = [f"User: {t.user}\nAssistant: {t.assistant}" for t in recent]
            parts.append("Most recent turns:\n" + "\n\n".join(lines))
        return "\n\n".join(parts) or "(no previous conversation)"
```

Langkah 6 — `agents/planner.py`

⏱️ ±25 menit

Penjelasan

Planner membuat **dua jenis keputusan**:

1. Sebelum menjawab: memakai LLM (Haiku, murah dan cepat). Outputnya schema `Plan`:

Field	Isi
`intent`	`medical_question`, `follow_up`, `summary_request`, `greeting`, `out_of_scope`, `emergency`
`language`	bahasa user ("Indonesian"/"English"), supaya jawaban memakai bahasa yang sama
`search_query`	**query bahasa Inggris yang berdiri sendiri**: "obatnya apa?" → "bronchitis treatment"
`direct_reply`	balasan langsung untuk sapaan / di luar topik / darurat
`reasoning`	satu kalimat alasan (tampil di agent trace)

💡 **Kenapa query rewriting sangat penting?** Bukunya berbahasa Inggris dan model embedding juga berbahasa Inggris. Pertanyaan "obat kencing manis" mentah-mentah tidak akan ketemu. Hasil evaluasi: Hit@5 pertanyaan mentah **74%** → setelah ditulis ulang Planner **100%**.

2. Setelah Evaluator menilai: memakai ATURAN (bukan LLM), di `after_evaluation()`:

Kondisi	Keputusan
`insufficient_context` & masih boleh mencoba	`requery`
`insufficient_context` & percobaan habis	`escalate`
confidence ≥ 0,70	`accept`
confidence < 0,70 & masih boleh mencoba	`requery`
percobaan habis, confidence 0,45–0,70	`accept` (dengan peringatan)
percobaan habis, confidence < 0,45	`escalate`

💡 **Kenapa aturan, bukan LLM?** Gratis, cepat, bisa diprediksi, dan bisa di-unit-test. LLM dipakai hanya di tempat yang benar-benar butuh pemahaman bahasa.

`route()` hanya mencatat agent mana yang akan aktif (untuk ditampilkan di trace). `should_summarize()` → True setiap 3 giliran.

Kode

File: `doctor_agent/agents/planner.py`

```
"""Planner Agent — the coordinator.

Two kinds of decisions:
1. BEFORE answering (LLM call, cheap model): what does the user want?
   -> intent, language, and a standalone English search query.
2. AFTER evaluation (deterministic policy): accept, re-query or escalate,
   based on the Evaluator's confidence score.  Rules (not an LLM) make this
   step predictable, testable and free.
"""
```

```python
from __future__ import annotations

from dataclasses import dataclass
from typing import Literal

from pydantic import BaseModel, Field

from config import ACCEPT_THRESHOLD, ESCALATE_THRESHOLD, MAX_ATTEMPTS, SUMMARY_EVERY
from doctor_agent.llm import LLM, Usage
from doctor_agent.memory import Memory

Intent = Literal["medical_question", "follow_up", "summary_request", "greeting", "out_of_scope",
"emergency"]


class Plan(BaseModel):
    intent: Intent
    language: str = Field(description="Language the user writes in, e.g. 'Indonesian' or 'English'")
    search_query: str = Field(description="Standalone English search query for the medical
encyclopedia; empty if no retrieval is needed")
    direct_reply: str = Field(description="Short reply for greeting / out_of_scope / emergency, in the
user's language; empty otherwise")
    reasoning: str = Field(description="One sentence explaining the decision")


@dataclass
class Decision:
    action: Literal["accept", "requery", "escalate"]
    reason: str


SYSTEM = """You are the Planner of a multi-agent medical assistant. The knowledge source is the
"Medical Book": the Gale Encyclopedia of Medicine, volume 1 (topics from A to B, written in English).

Classify the user's latest message:
- medical_question: a new health/medical question.
- follow_up: a medical question that depends on earlier turns ("how is it treated?").
- summary_request: the user asks to summarise / recap the conversation.
- greeting: greetings, thanks, small talk, questions about what you can do.
- out_of_scope: clearly NOT about health or medicine (sports, coding, politics...).
- emergency: the user describes an acute, possibly life-threatening situation happening now
  (e.g. chest pain with shortness of breath, stroke signs, heavy bleeding, overdose, suicidal intent).

Rules:
- Any health topic is medical_question/follow_up even if it might not be in the book;
  the retrieval step decides whether the book covers it. Do NOT answer medical questions yourself.
- search_query: rewrite the question as a standalone English search query, resolving pronouns from
  the conversation ("it" -> the disease being discussed) and translating medical terms into the
  terminology an English encyclopedia uses. Empty for greeting/out_of_scope/summary_request.
- direct_reply: for greeting, a friendly 1-2 sentence reply explaining you answer questions from the
  Medical Book; for out_of_scope, politely say you only handle medical topics; for emergency, tell
  the user to contact emergency services immediately (Indonesia: 119 or 112, or the nearest ER).
  Write it in the user's language. Empty for the other intents."""


class PlannerAgent:
    name = "planner"

    def __init__(self, llm: LLM):
        self.llm = llm

    # ------------------------------------------------------------ 1. plan
    def plan(self, message: str, memory: Memory) -> tuple[Plan, Usage]:
        prompt = f"{memory.context()}\n\nLatest user message:\n{message}"
        return self.llm.json(self.name, SYSTEM, [{"role": "user", "content": prompt}], Plan)

    @staticmethod
    def route(plan: Plan) -> list[str]:
```

```
        """Which agents to activate for this intent."""
        return {
            "medical_question": ["rag", "evaluator", "summarizer?"],
            "follow_up": ["rag", "evaluator", "summarizer?"],
            "summary_request": ["summarizer"],
            "greeting": [],
            "out_of_scope": [],
            "emergency": [],
        }[plan.intent]

    # ------------------------------------------------------------ 2. decide
    @staticmethod
    def after_evaluation(verdict: str, confidence: float, attempt: int) -> Decision:
        can_retry = attempt < MAX_ATTEMPTS
        if verdict == "insufficient_context":
            if can_retry:
                return Decision("requery", "retrieved passages do not answer the question; trying a
better query")
            return Decision("escalate", "the Medical Book does not seem to cover this question")
        if confidence >= ACCEPT_THRESHOLD:
            return Decision("accept", f"confidence {confidence:.2f} >= {ACCEPT_THRESHOLD}")
        if can_retry:
            return Decision("requery", f"confidence {confidence:.2f} < {ACCEPT_THRESHOLD}; re-
querying")
        if confidence >= ESCALATE_THRESHOLD:
            return Decision("accept", f"accepted with low confidence {confidence:.2f} (shown with a
warning)")
        return Decision("escalate", f"confidence {confidence:.2f} < {ESCALATE_THRESHOLD} after re-
query")

    @staticmethod
    def should_summarize(memory: Memory) -> bool:
        return memory.turns_since_summary >= SUMMARY_EVERY
```

Langkah 7 — `agents/rag.py`

⏱️ ±15 menit

Penjelasan

1. `retriever.search(search_query)` → 5 potongan teratas.
2. `format_context()` → menomori potongan: `[1] Acne - Causes and symptoms (p. 25) Acne vulgaris affects ...`
3. Prompt berisi: memori + potongan bernomor + (kalau ini percobaan ulang) **feedback dari Evaluator** + pertanyaan.
4. System prompt memuat aturan kunci: **hanya pakai potongan**, **wajib sitasi [n]**, kalau tidak ada katakan tidak ada, jawab dalam bahasa user, maksimal ±200 kata, sarankan ke dokter untuk pengobatan.

`RagResult.top_similarity` = cosine tertinggi, ditampilkan di trace sebagai sinyal kualitas retrieval.

💡 **Kenapa wajib sitasi?** Supaya setiap klaim bisa dilacak ke halaman buku, dan Evaluator bisa mengecek klaim terhadap potongan yang dirujuk.

Kode

File: `doctor_agent/agents/rag.py`

```
"""RAG Agent - retrieves passages from the Medical Book and writes a grounded,
cited answer.  It is told to use ONLY the passages, so anything it says can
be traced back to a page of the book (and checked by the Evaluator)."""
from __future__ import annotations

from dataclasses import dataclass

from config import TOP_K
from doctor_agent.llm import LLM, Usage
from doctor_agent.memory import Memory
from doctor_agent.retriever import Hit, HybridRetriever

SYSTEM = """You are a careful medical information assistant. You answer ONLY with information found in
the numbered excerpts from the "Medical Book" (Gale Encyclopedia of Medicine) given to you.

Rules:
1. Use only the excerpts. Do not add facts from your own knowledge, even if you are sure they are true.
2. Cite the excerpt numbers after each claim, like [1] or [2][3].
3. If the excerpts do not contain the answer, say plainly that the Medical Book does not cover it
   (you may mention what related information the excerpts do contain). Never guess.
4. Write in {language}. Translate the content faithfully; keep medical terms understandable.
5. Be clear, calm and empathetic. At most ~200 words; short paragraphs or bullet points.
6. When you describe treatments, drugs or dosages, end with one short sentence advising the user
   to consult a doctor for their personal situation."""


def format_context(hits: list[Hit]) -> str:
    blocks = []
    for n, h in enumerate(hits, 1):
        c = h.chunk
        page = f"p. {c['book_page']}" if c["book_page"] else f"PDF p. {c['pdf_page']}"
        blocks.append(f"[{n}] {c['entry']} - {c['section']} ({page})\n{c['text']}")
    return "\n\n".join(blocks)
```

```python
@dataclass
class RagResult:
    answer: str
    hits: list[Hit]
    query: str
    usage: Usage

    @property
    def top_similarity(self) -> float:
        return max((h.dense_sim for h in self.hits), default=0.0)


class RAGAgent:
    name = "rag"

    def __init__(self, llm: LLM, retriever: HybridRetriever):
        self.llm = llm
        self.retriever = retriever

    def run(self, question: str, search_query: str, language: str, memory: Memory,
            feedback: str = "") -> RagResult:
        hits = self.retriever.search(search_query or question, TOP_K)
        prompt = (
            f"{memory.context()}\n\n"
            f"Excerpts from the Medical Book:\n{format_context(hits)}\n\n"
            + (f"A reviewer found problems with a previous attempt - avoid them:\n{feedback}\n\n" if
feedback else "")
            + f"User question: {question}"
        )
        answer, usage = self.llm.text(self.name, SYSTEM.format(language=language),
                                      [{"role": "user", "content": prompt}])
        return RagResult(answer, hits, search_query, usage)
```

Langkah 8 — `agents/evaluator.py`

⏱️ ±20 menit

Penjelasan

Evaluator adalah **LLM-as-judge**: LLM kedua yang memeriksa jawaban RAG terhadap **potongan yang sama**.

Prosedur di system prompt:

1. Pecah jawaban menjadi klaim-klaim. Klaim dianggap didukung hanya jika ada di potongan (parafrase/terjemahan boleh). Fakta yang benar secara medis tapi tidak ada di potongan tetap dihitung **tidak didukung**.
2. Pilih verdict: - `pass` : semua klaim didukung - `revise` : ada klaim tak didukung / poin penting hilang → **tulis** `revised_answer` - `insufficient_context` : potongan tidak menjawab pertanyaan → beri `suggested_query`
3. Beri skor 0–1 untuk **jawaban final** (asli kalau pass, revisi kalau revise).

Confidence dihitung di kode, bukan ditebak LLM:

```
confidence = 0,5 × faithfulness + 0,3 × completeness + 0,2 × relevance
(maksimal 0,4 jika insufficient_context)
```

💡 **Kenapa faithfulness bobotnya terbesar?** Di domain medis, jawaban **salah** lebih berbahaya daripada jawaban **kurang lengkap**.

Kode

File: `doctor_agent/agents/evaluator.py`

```python
"""Evaluator Agent — an LLM-as-judge that fact-checks the RAG answer against
the same excerpts the RAG agent saw.

It scores faithfulness (no hallucination), completeness, relevance and tone,
lists unsupported claims, and — if needed — returns a corrected answer.
The final confidence score is a fixed weighted mix of those scores
(computed in code, so it is transparent and reproducible)."""
from __future__ import annotations

from typing import Literal

from pydantic import BaseModel, Field

from config import CONFIDENCE_WEIGHTS
from doctor_agent.agents.rag import format_context
from doctor_agent.llm import LLM, Usage
from doctor_agent.retriever import Hit


class Evaluation(BaseModel):
    faithfulness: float = Field(description="0-1: share of claims in the FINAL answer supported by the excerpts")
    completeness: float = Field(description="0-1: how fully the FINAL answer addresses the question, given what the excerpts contain")
```

```
    relevance: float = Field(description="0-1: how well the excerpts and the FINAL answer match the
question")
    tone: float = Field(description="0-1: clear, calm, empathetic, non-alarming, appropriate for a
medical audience")
    unsupported_claims: list[str] = Field(description="Claims in the ORIGINAL answer not supported by
the excerpts")
    missing_points: list[str] = Field(description="Important points present in the excerpts but missing
from the answer")
    verdict: Literal["pass", "revise", "insufficient_context"]
    revised_answer: str = Field(description="Corrected answer if verdict is 'revise', otherwise empty")
    suggested_query: str = Field(description="Better English search query if verdict is
'insufficient_context', otherwise empty")
    feedback: str = Field(description="One or two sentences summarising the review")


SYSTEM = """You are the Evaluator in a medical question-answering system. You fact-check an answer
against the numbered excerpts from the Medical Book that the answer was supposed to be based on.

Procedure:
1. Split the answer into individual claims. A claim is supported only if an excerpt states it
   (paraphrase and translation are fine). Anything else is unsupported - even if medically true.
2. Choose a verdict:
   - pass: every claim is supported and the answer addresses the question well.
   - revise: the excerpts do answer the question, but the answer has unsupported claims, wrong
     citations, missing key points, or poor tone. Write `revised_answer`: same language, same style,
     unsupported claims removed, key missing points added from the excerpts, citations [n] kept.
   - insufficient_context: the excerpts do not contain the information needed to answer the question.
     Give `suggested_query`: a better English query for the encyclopedia (or empty if the topic is
     probably not in an A-B volume at all). An answer that correctly says "the book does not cover
     this" is still insufficient_context.
3. Score the FINAL answer - the original if verdict is pass, otherwise your revised_answer -
   on faithfulness, completeness, relevance and tone (0.0-1.0). Be strict and calibrated:
   0.9+ only for excellent, fully supported answers. For insufficient_context, completeness <= 0.3."""


class EvaluatorAgent:
    name = "evaluator"

    def __init__(self, llm: LLM):
        self.llm = llm

    def run(self, question: str, answer: str, hits: list[Hit], language: str) -> tuple[Evaluation,
float, Usage]:
        prompt = (
            f"User question ({language}): {question}\n\n"
            f"Excerpts from the Medical Book:\n{format_context(hits)}\n\n"
            f"Answer to evaluate:\n{answer}"
        )
        ev, usage = self.llm.json(self.name, SYSTEM, [{"role": "user", "content": prompt}], Evaluation)
        return ev, confidence(ev), usage


def confidence(ev: Evaluation) -> float:
    clamp = lambda x: max(0.0, min(1.0, float(x)))
    score = sum(w * clamp(getattr(ev, k)) for k, w in CONFIDENCE_WEIGHTS.items())
    if ev.verdict == "insufficient_context":
        score = min(score, 0.4)  # never "confident" when the book lacks the answer
    return round(score, 3)
```

Langkah 9 — `agents/summarizer.py`

⏱️ ±10 menit

Penjelasan

Dua fungsi:

- `update(memory)`: setiap 3 giliran, gabungkan ringkasan lama + giliran baru menjadi ringkasan ≤ 120 kata (bahasa Inggris, karena dipakai internal oleh agent lain). Lalu reset penghitung.
- `for_user(memory, language)`: kalau user minta "rangkum percakapan kita", buat 3–6 poin dalam bahasa user.

Kode

File: `doctor_agent/agents/summarizer.py`

```
"""Summarizer Agent — condenses the conversation.

* update():   every N turns, folds the newest turns into a running summary
              (the bot's long-term memory, injected into later prompts).
* for_user(): when the user asks "summarise our chat", writes a readable recap."""
from __future__ import annotations

from doctor_agent.llm import LLM, Usage
from doctor_agent.memory import Memory

MEMORY_SYSTEM = """You maintain the memory of a medical Q&A chat. Merge the previous summary and the
new
turns into ONE updated summary of at most 120 words, in English. Keep: topics/diseases discussed,
key facts already given, personal context the user shared (age, symptoms, conditions), and open
questions. Drop greetings and filler. Output only the summary text."""

USER_SYSTEM = """Write a short recap of this medical Q&A conversation for the user, in {language}.
Use 3-6 bullet points covering the topics asked about and the main information given.
Do not add new medical facts. Output only the recap."""


def _transcript(memory: Memory, turns) -> str:
    lines = [f"User: {t.user}\nAssistant: {t.assistant}" for t in turns]
    prev = memory.summary or "(none)"
    return f"Previous summary:\n{prev}\n\nNew turns:\n" + "\n\n".join(lines)


class SummarizerAgent:
    name = "summarizer"

    def __init__(self, llm: LLM):
        self.llm = llm

    def update(self, memory: Memory) -> Usage:
        text, usage = self.llm.text(self.name, MEMORY_SYSTEM,
                                    [{"role": "user", "content": _transcript(memory,
memory.unsummarized())}])
        memory.summary = text
        memory.turns_since_summary = 0
        return usage

    def for_user(self, memory: Memory, language: str) -> tuple[str, Usage]:
        if not memory.turns:
```

```
            return "", None  # nothing to summarise
        content = _transcript(memory, memory.unsummarized() or memory.recent())
        return self.llm.text(self.name, USER_SYSTEM.format(language=language),
                             [{"role": "user", "content": content}])
```

Langkah 10 — `orchestrator.py`: menyambung semua agent

⏱️ ±40 menit

Penjelasan

`DoctorAgentSystem.ask(message, memory)` menjalankan **satu giliran**:

```
1. plan = Planner.plan()                    → log Step
2a. greeting / out_of_scope / emergency     → jawaban = plan.direct_reply
2b. summary_request                         → Summarizer.for_user()
2c. medical / follow_up                     → _answer_medical()
3. memory.add(); kalau sudah 3 giliran      → Summarizer.update()
```

`_answer_medical()` berisi loop **maksimal 2 percobaan**:

```
untuk attempt 1..2:
    rag  = RAG.run(query, feedback)
    ev   = Evaluator.run(rag.answer)
    final = ev.revised_answer kalau verdict "revise", selain itu rag.answer
    simpan sebagai "best" kalau confidence-nya tertinggi
    keputusan = Planner.after_evaluation()
    kalau bukan "requery" → berhenti
    query    = ev.suggested_query          ← percobaan ulang pakai saran Evaluator
    feedback = klaim tak didukung + poin yang hilang
keputusan final dibuat dari percobaan TERBAIK
escalate → pesan "konsultasikan ke dokter" (+ jawaban parsial kalau ada)
```

Objek pendukung:

- `Step`: satu baris di *agent trace* (agent, aksi, detail, detik, token, biaya, model)
- `TurnResult`: semua yang dibutuhkan UI: jawaban, intent, confidence, evaluasi, sumber, langkah, escalated/revised, jumlah percobaan, total biaya & token
- `on_step`: *callback* supaya UI bisa menampilkan progres agent secara langsung
- `escalation_message()`: pesan eskalasi dalam bahasa Indonesia atau Inggris

💡 **Kenapa tidak pakai LangChain/LangGraph?** Alurnya sederhana dan deterministik. Python biasa lebih transparan, mudah di-test (lihat `FakeLLM` di Langkah 12), dan tanpa dependensi berat.

Kode

File: `doctor_agent/orchestrator.py`

```
"""Orchestrator - runs one user turn through the agents.

    user message
        -> Planner.plan()            intent + search query          (Haiku)
        -> [medical] loop up to MAX_ATTEMPTS:
               RAG.run()             retrieve + grounded answer     (Opus)
               Evaluator.run()       fact-check, score, revise      (Opus)
               Planner.after_evaluation()  accept / requery / escalate (rules)
        -> [summary_request] Summarizer.for_user()
        -> memory.add(); every N turns Summarizer.update()          (Haiku)

Every agent call is logged as a Step, which the UI shows as an "agent trace".
```

```
"""
from __future__ import annotations

import time
from dataclasses import dataclass, field
from typing import Callable

from config import MAX_ATTEMPTS
from doctor_agent.agents.evaluator import Evaluation, EvaluatorAgent
from doctor_agent.agents.planner import Plan, PlannerAgent
from doctor_agent.agents.rag import RAGAgent
from doctor_agent.agents.summarizer import SummarizerAgent
from doctor_agent.llm import LLM, LLMRefusal, Usage
from doctor_agent.memory import Memory
from doctor_agent.retriever import Hit, HybridRetriever


@dataclass
class Step:
    agent: str
    action: str
    detail: str = ""
    seconds: float = 0.0
    input_tokens: int = 0
    output_tokens: int = 0
    cost: float = 0.0
    model: str = ""

    @classmethod
    def from_usage(cls, u: Usage | None, action: str, detail: str = "") -> "Step":
        if u is None:
            return cls("system", action, detail)
        return cls(u.agent, action, detail, u.seconds, u.input_tokens, u.output_tokens, u.cost,
u.model)


@dataclass
class TurnResult:
    answer: str
    intent: str
    plan: Plan | None = None
    confidence: float | None = None
    evaluation: Evaluation | None = None
    sources: list[Hit] = field(default_factory=list)
    steps: list[Step] = field(default_factory=list)
    escalated: bool = False
    revised: bool = False
    attempts: int = 0
    seconds: float = 0.0

    @property
    def cost(self) -> float:
        return sum(s.cost for s in self.steps)

    @property
    def tokens(self) -> tuple[int, int]:
        return sum(s.input_tokens for s in self.steps), sum(s.output_tokens for s in self.steps)


def _is_indonesian(language: str) -> bool:
    return "indones" in language.lower() or "bahasa" in language.lower()


def escalation_message(language: str, partial: str = "") -> str:
    if _is_indonesian(language):
        msg = ("⚠️ **Saya belum bisa menjawab ini dengan yakin dari Medical Book.** "
               "Informasi yang relevan tidak ditemukan atau tidak cukup di dalam buku. "
               "Silakan konsultasikan pertanyaan ini dengan dokter atau tenaga kesehatan.")
```

```
        prefix = "Informasi terkait yang ditemukan (tingkat keyakinan rendah):"
    else:
        msg = ("⚠️ **I can't answer this confidently from the Medical Book.** "
               "The relevant information was not found or is insufficient in the book. "
               "Please consult a doctor or a qualified health professional.")
        prefix = "Related information that was found (low confidence):"
    return f"{msg}\n\n{prefix}\n\n{partial}" if partial else msg


class DoctorAgentSystem:
    def __init__(self, llm: LLM | None = None, retriever: HybridRetriever | None = None):
        self.llm = llm or LLM()
        self.retriever = retriever or HybridRetriever.load()
        self.planner = PlannerAgent(self.llm)
        self.rag = RAGAgent(self.llm, self.retriever)
        self.evaluator = EvaluatorAgent(self.llm)
        self.summarizer = SummarizerAgent(self.llm)

    def ask(self, message: str, memory: Memory, on_step: Callable[[Step], None] | None = None) ->
TurnResult:
        t0 = time.perf_counter()
        steps: list[Step] = []

        def log(step: Step) -> None:
            steps.append(step)
            if on_step:
                on_step(step)

        # 1. Planner decides what to do ------------------------------------------
        plan, u = self.planner.plan(message, memory)
        route = self.planner.route(plan)
        log(Step.from_usage(u, f"intent = {plan.intent}",
                            f"route: {' → '.join(route) or 'direct reply'} | query: {plan.search_query
or '—'} | {plan.reasoning}"))

        result = TurnResult(answer="", intent=plan.intent, plan=plan, steps=steps)

        if plan.intent in ("greeting", "out_of_scope", "emergency"):
            result.answer = plan.direct_reply
            result.escalated = plan.intent == "emergency"
        elif plan.intent == "summary_request":
            text, u = self.summarizer.for_user(memory, plan.language)
            log(Step.from_usage(u, "recap for user"))
            result.answer = text or ("Belum ada percakapan untuk dirangkum." if
_is_indonesian(plan.language)
                                     else "There is no conversation to summarise yet.")
        else:
            self._answer_medical(message, plan, memory, result, log)

        # 2. Memory + periodic summarisation -------------------------------------
        memory.add(message, result.answer)
        if self.planner.should_summarize(memory):
            u = self.summarizer.update(memory)
            log(Step.from_usage(u, "memory updated", memory.summary[:160]))

        result.seconds = time.perf_counter() - t0
        return result

    def _answer_medical(self, message: str, plan: Plan, memory: Memory, result: TurnResult, log) ->
None:
        query, feedback = plan.search_query or message, ""
        best = None  # (confidence, final_answer, evaluation, hits, revised)

        for attempt in range(1, MAX_ATTEMPTS + 1):
            result.attempts = attempt
            try:
                rag = self.rag.run(message, query, plan.language, memory, feedback)
            except LLMRefusal as e:
```

```
                log(Step("rag", "refused", str(e)))
                break
            sources = ", ".join(dict.fromkeys(h.chunk["entry"] for h in rag.hits))
            log(Step.from_usage(rag.usage, f"attempt {attempt}: answered",
                                f"query: {query} | top sim {rag.top_similarity:.2f} | sources:
{sources}"))

            ev, conf, u = self.evaluator.run(message, rag.answer, rag.hits, plan.language)
            log(Step.from_usage(u, f"verdict = {ev.verdict}, confidence = {conf:.2f}",
                                f"F {ev.faithfulness:.2f} · C {ev.completeness:.2f} · R
{ev.relevance:.2f} · "
                                f"T {ev.tone:.2f} | unsupported: {len(ev.unsupported_claims)} |
{ev.feedback}"))

            revised = ev.verdict == "revise" and bool(ev.revised_answer.strip())
            final = ev.revised_answer.strip() if revised else rag.answer
            if best is None or conf > best[0]:
                best = (conf, final, ev, rag.hits, revised)

            decision = self.planner.after_evaluation(ev.verdict, conf, attempt)
            log(Step("planner", f"decision = {decision.action}", decision.reason))
            if decision.action != "requery":
                break
            # re-query: use the Evaluator's suggested query and tell RAG what went wrong
            query = ev.suggested_query.strip() or f"{plan.search_query} {message}"
            feedback = "; ".join(ev.unsupported_claims + [f"missing: {m}" for m in ev.missing_points])
or ev.feedback

        if best is None:  # the model refused on the first attempt
            result.answer, result.escalated = escalation_message(plan.language), True
            return

        conf, answer, ev, hits, revised = best
        result.confidence, result.evaluation, result.sources, result.revised = conf, ev, hits, revised
        # final call is made on the BEST attempt, with no retries left
        final = self.planner.after_evaluation(ev.verdict, conf, MAX_ATTEMPTS)
        if result.attempts > 1:
            log(Step("planner", f"final = {final.action}", f"best attempt: {final.reason}"))
        if final.action == "escalate":
            result.escalated = True
            partial = answer if ev.verdict != "insufficient_context" else ""
            result.answer = escalation_message(plan.language, partial)
        else:
            result.answer = answer
```

Script uji cepat: `scripts/smoke_test.py`

File: `scripts/smoke_test.py`

```
"""Quick manual check of the whole pipeline from the terminal (uses the real API)."""
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from doctor_agent.memory import Memory
from doctor_agent.orchestrator import DoctorAgentSystem

QUESTIONS = sys.argv[1:] or [
    "What causes acne?",
    "Apa saja gejala bronkitis?",
    "Bagaimana cara mengobatinya?",
    "Apa obat untuk diabetes tipe 2?",
]

system = DoctorAgentSystem()
```

```
memory = Memory()
total = 0.0
for q in QUESTIONS:
    r = system.ask(q, memory)
    total += r.cost
    print("=" * 90)
    print("Q:", q)
    for s in r.steps:
        print(f"  [{s.agent:10}] {s.action:45} {s.seconds:5.1f}s ${s.cost:.4f}  {s.detail[:150]}")
    conf = f"{r.confidence:.2f}" if r.confidence is not None else "–"
    print(f"-> intent={r.intent} confidence={conf} escalated={r.escalated} revised={r.revised} "
          f"attempts={r.attempts} {r.seconds:.1f}s ${r.cost:.4f}")
    print("A:", r.answer[:700])
print("=" * 90)
print(f"memory summary: {memory.summary!r}")
print(f"TOTAL COST: ${total:.4f}")
```

✅ Checkpoint (memanggil API, ±$0,20)

```
python scripts/smoke_test.py
```

Yang harus terlihat:

- "What causes acne?" → confidence ≥ 0,9, sumber *Acne*
- "Apa saja gejala bronkitis?" → jawaban bahasa Indonesia
- "Bagaimana cara mengobatinya?" → intent `follow_up`, query `bronchitis treatment`
- Setelah 3 pertanyaan: baris `[summarizer] memory updated`

Tes satu pertanyaan saja:

```
python scripts/smoke_test.py "What are the symptoms of dengue fever?"
# harus: escalated=True, attempts=2
```

Langkah 11 — `app.py`: UI Streamlit

⏱️ ±40 menit · file: `app.py` (di root folder)

Konsep Streamlit yang dipakai

Konsep	Kegunaan di sini

| Script dijalankan ulang dari atas setiap ada interaksi | karena itu state disimpan di `st.session_state` | | `st.session_state` | menyimpan `memory`, `messages`, total biaya per sesi browser | | `@st.cache_resource` | `DoctorAgentSystem` (index + model embedding) dimuat **sekali** saja, bukan setiap rerun | | `st.chat_message` / `st.chat_input` | gelembung chat dan kotak input | | `st.status` | kotak "Agents are working..." yang diisi langkah agent secara live lewat `on_step` | | `:green-badge[...]` | badge berwarna di markdown (confidence, escalated, revised) | | `st.expander` | panel lipat: Evaluator feedback, Sources, Agent trace | | `st.rerun()` | menggambar ulang halaman supaya angka di sidebar ter-update |

Alur `app.py`

1. `init_state()`: siapkan memory & riwayat pesan.
2. `sidebar()`: info agent & model, biaya sesi, ringkasan memori, tombol *Clear conversation*.
3. Cek API key; kalau tidak ada, tampilkan error lalu berhenti.
4. Gambar ulang semua pesan lama (+ `render_details` untuk jawaban asisten).
5. Kalau belum ada pesan, tampilkan tombol contoh pertanyaan.
6. Saat user mengirim pertanyaan → `system.ask(prompt, memory, on_step=...)` → tampilkan jawaban, simpan ke riwayat, `st.rerun()`.

Kode

File: `app.py`

```
"""Doctor Agent — Streamlit UI for the multi-agent Medical Book assistant.

Run:  streamlit run app.py
"""
import os

import streamlit as st

from config import ACCEPT_THRESHOLD, MODELS
from doctor_agent.memory import Memory
from doctor_agent.orchestrator import DoctorAgentSystem, Step, TurnResult

st.set_page_config(page_title="Doctor Agent", page_icon="🩺", layout="wide")

AGENT_ICON = {"planner": "🧭", "rag": "📚", "evaluator": "🔍", "summarizer": "📝", "system": "⚙️"}
AGENT_ROLE = {
    "planner": "Classifies intent, rewrites the query, decides accept / re-query / escalate",
    "rag": "Hybrid retrieval (FAISS + BM25) and a grounded, cited answer",
    "evaluator": "Fact-checks the answer, scores it, revises if needed",
    "summarizer": "Keeps a running conversation summary as memory",
}
EXAMPLES = [
    "What causes acne?",
```

```
    "Apa saja gejala bronkitis?",
    "Bagaimana cara mengobatinya?",
    "How is malaria treated?",
    "Rangkum percakapan kita",
]


# ---------------------------------------------------------------------- state
@st.cache_resource(show_spinner="Loading Medical Book index...")
def load_system() -> DoctorAgentSystem:
    return DoctorAgentSystem()


def init_state() -> None:
    st.session_state.setdefault("memory", Memory())
    st.session_state.setdefault("messages", [])  # {"role", "content", "result"}
    st.session_state.setdefault("total_cost", 0.0)
    st.session_state.setdefault("total_tokens", 0)


def reset() -> None:
    for k in ("memory", "messages", "total_cost", "total_tokens"):
        st.session_state.pop(k, None)


# ---------------------------------------------------------------------- rendering helpers
def confidence_badges(r: TurnResult) -> str:
    badges = []
    if r.confidence is not None:
        color = "green" if r.confidence >= ACCEPT_THRESHOLD else "orange" if not r.escalated else "red"
        badges.append(f":{color}-badge[Confidence {r.confidence:.0%}]")
    if r.escalated:
        badges.append(":red-badge[Escalated]")
    if r.revised:
        badges.append(":violet-badge[Revised by Evaluator]")
    if r.attempts > 1:
        badges.append(f":blue-badge[Re-queried ×{r.attempts - 1}]")
    badges.append(f":gray-badge[{r.intent.replace('_', ' ')}]")
    tin, tout = r.tokens
    badges.append(f":gray-badge[{r.seconds:.1f}s · {tin + tout:,} tokens · ${r.cost:.4f}]")
    return " ".join(badges)


def render_details(r: TurnResult) -> None:
    st.markdown(confidence_badges(r))
    if r.evaluation is not None:
        ev = r.evaluation
        with st.expander(f"🔍 Evaluator feedback — verdict: **{ev.verdict}**"):
            cols = st.columns(4)
            for col, (label, val) in zip(cols, [("Faithfulness", ev.faithfulness), ("Completeness",
ev.completeness),
                                                ("Relevance", ev.relevance), ("Tone", ev.tone)]):
                col.metric(label, f"{val:.2f}")
            st.write(ev.feedback)
            if ev.unsupported_claims:
                st.markdown("**Unsupported claims found in the draft** (removed or flagged):")
                for c in ev.unsupported_claims:
                    st.markdown(f"- {c}")
            if ev.missing_points:
                st.markdown("**Missing points:**")
                for c in ev.missing_points:
                    st.markdown(f"- {c}")
    if r.sources:
        with st.expander(f"📚 Sources from the Medical Book ({len(r.sources)})"):
            for n, h in enumerate(r.sources, 1):
                c = h.chunk
                page = c["book_page"] or f"PDF {c['pdf_page']}"
                st.markdown(f"**[{n}] {c['entry']} — {c['section']}** · page {page} · similarity
```

```
{h.dense_sim:.2f}")
                st.caption(c["text"][:400] + ("…" if len(c["text"]) > 400 else ""))
    with st.expander(f"🧭 Agent trace ({len(r.steps)} steps)"):
        st.dataframe(
            [{"agent": f"{AGENT_ICON.get(s.agent, '')} {s.agent}", "action": s.action, "model":
s.model,
              "time (s)": round(s.seconds, 2), "tokens in/out": f"{s.input_tokens}/{s.output_tokens}",
              "cost ($)": round(s.cost, 5), "detail": s.detail} for s in r.steps],
            hide_index=True, width="stretch",
        )


def sidebar() -> None:
    with st.sidebar:
        st.header("🩺 Doctor Agent")
        st.caption("Multi-agent RAG assistant over the **Medical Book** "
                   "(Gale Encyclopedia of Medicine, vol. 1 — topics A–B).")
        st.subheader("Agents")
        for name, model in MODELS.items():
            st.markdown(f"{AGENT_ICON[name]} **{name.capitalize()}** · `{model}`")
            st.caption(AGENT_ROLE[name])
        st.subheader("Session")
        c1, c2 = st.columns(2)
        c1.metric("Questions", sum(m["role"] == "user" for m in st.session_state.messages))
        c2.metric("Cost", f"${st.session_state.total_cost:.3f}")
        st.caption(f"{st.session_state.total_tokens:,} tokens used")
        st.subheader("Memory (Summarizer)")
        st.caption(st.session_state.memory.summary or "No summary yet — created every 3 turns.")
        st.button("🗑️ Clear conversation", on_click=reset, width="stretch")
        st.caption("⚠️ Educational tool, not a substitute for professional medical advice.")


# ---------------------------------------------------------------------- main
init_state()
sidebar()

st.title("🩺 Doctor Agent")
st.caption("Ask a medical question in English or Indonesian. Four agents collaborate: "
           "Planner → RAG → Evaluator → Summarizer.")

if not os.environ.get("ANTHROPIC_API_KEY"):
    st.error("ANTHROPIC_API_KEY is not set. Copy `.env.example` to `.env` and add your key, then
restart.")
    st.stop()

system = load_system()

for m in st.session_state.messages:
    with st.chat_message(m["role"], avatar="🩺" if m["role"] == "assistant" else None):
        st.markdown(m["content"])
        if m.get("result"):
            render_details(m["result"])

if not st.session_state.messages:
    st.markdown("**Try one of these:**")
    cols = st.columns(len(EXAMPLES))
    for col, ex in zip(cols, EXAMPLES):
        if col.button(ex, width="stretch"):
            st.session_state.pending = ex
            st.rerun()

prompt = st.chat_input("Ask about a disease, symptom, test or treatment...") or
st.session_state.pop("pending", None)
if prompt:
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)
```

```
    with st.chat_message("assistant", avatar="🩺"):
        with st.status("Agents are working...", expanded=True) as status:
            def on_step(s: Step) -> None:
                st.write(f"{AGENT_ICON.get(s.agent, '')} **{s.agent.capitalize()}** — {s.action}")

            try:
                result = system.ask(prompt, st.session_state.memory, on_step=on_step)
            except Exception as e:  # show API / network errors instead of crashing the app
                status.update(label="Something went wrong", state="error")
                st.error(f"{type(e).__name__}: {e}")
                st.session_state.messages.pop()
                st.stop()
            status.update(label=f"Done in {result.seconds:.1f}s", state="complete", expanded=False)
        st.markdown(result.answer)
        render_details(result)

    st.session_state.messages.append({"role": "assistant", "content": result.answer, "result": result})
    st.session_state.total_cost += result.cost
    st.session_state.total_tokens += sum(result.tokens)
    st.rerun()  # refresh sidebar counters
```

✅ Checkpoint

```
streamlit run app.py
```

Buka `http://localhost:8501`, lalu coba:

1. Klik tombol "What causes acne?" → jawaban + badge hijau *Confidence ~95%*
2. Buka panel *Agent trace* → 4 langkah (planner, rag, evaluator, planner)
3. "Apa gejala demam berdarah dengue?" → badge merah *Escalated* + *Re-queried ×1*
4. "Rangkum percakapan kita" → rekap poin-poin

Hentikan dengan `Ctrl + C`.

Langkah 12 — Unit test (opsional)

⏱ ±15 menit · file: `tests/test_core.py`

Test ini **tidak memanggil API** (gratis). Triknya adalah `FakeLLM`: LLM palsu yang mengembalikan jawaban yang sudah kita skenariokan, sehingga alur orchestrator (terima, revisi, cari ulang lalu eskalasi, sapaan, Summarizer tiap 3 giliran) bisa dites dengan pasti.

Kelompok test	Yang dicek
chunking	potongan pendek tidak dipecah; potongan panjang dipecah sesuai ukuran dan ada overlap
evaluator	rumus confidence; batas 0,4 untuk insufficient context; schema ketat
planner policy	6 skenario tabel keputusan
orchestration	alur terima / revisi / cari ulang→eskalasi / sapaan / Summarizer
retrieval	entri yang benar ditemukan (termasuk regresi bug "ALS/palsy")

File: `tests/test_core.py`

```
"""Offline tests (no API calls, no cost): chunking, planner policy and the
orchestration flow, using a scripted fake LLM."""
import sys
from pathlib import Path

import pytest

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from config import ACCEPT_THRESHOLD, INDEX_DIR, MAX_ATTEMPTS
from doctor_agent.agents.evaluator import Evaluation, confidence
from doctor_agent.agents.planner import Plan, PlannerAgent
from doctor_agent.ingest import split_text
from doctor_agent.llm import Usage, _strict_schema
from doctor_agent.memory import Memory
from doctor_agent.orchestrator import DoctorAgentSystem
from doctor_agent.retriever import Hit


# ---------------------------------------------------------------- chunking
def test_split_short_text_is_one_chunk():
    assert split_text("Short text.", size=100) == ["Short text."]


def test_split_respects_size_and_overlaps():
    text = " ".join(f"Sentence number {i} is here." for i in range(200))
    parts = split_text(text, size=300, overlap=60)
    assert len(parts) > 5
    assert all(len(p) <= 300 + 60 for p in parts)
    # consecutive chunks share some words (overlap)
    assert set(parts[0].split()[-5:]) & set(parts[1].split()[:15])


# ---------------------------------------------------------------- evaluator scoring
def _ev(**kw) -> Evaluation:
    base = dict(faithfulness=1, completeness=1, relevance=1, tone=1, unsupported_claims=[],
                missing_points=[], verdict="pass", revised_answer="", suggested_query="",
feedback="ok")
    return Evaluation(**{**base, **kw})


def test_confidence_is_weighted_mix():
```

```
    assert confidence(_ev()) == 1.0
    assert confidence(_ev(faithfulness=0.0)) == 0.5


def test_confidence_capped_when_context_insufficient():
    assert confidence(_ev(verdict="insufficient_context")) <= 0.4


def test_strict_schema_requires_all_fields():
    schema = _strict_schema(Evaluation)
    assert schema["additionalProperties"] is False
    assert set(schema["required"]) == set(Evaluation.model_fields)


# ---------------------------------------------------------------------- planner policy
@pytest.mark.parametrize("verdict,conf,attempt,expected", [
    ("pass", 0.90, 1, "accept"),
    ("revise", 0.60, 1, "requery"),
    ("revise", 0.60, MAX_ATTEMPTS, "accept"),      # mid confidence, no retries left -> warn
    ("revise", 0.30, MAX_ATTEMPTS, "escalate"),
    ("insufficient_context", 0.30, 1, "requery"),
    ("insufficient_context", 0.30, MAX_ATTEMPTS, "escalate"),
])
def test_planner_policy(verdict, conf, attempt, expected):
    assert PlannerAgent.after_evaluation(verdict, conf, attempt).action == expected


# ---------------------------------------------------------------------- orchestration (fake LLM)
class FakeLLM:
    """Returns scripted outputs per agent, in order."""

    def __init__(self, script: dict[str, list]):
        self.script = {k: list(v) for k, v in script.items()}
        self.calls: list[str] = []

    def _next(self, agent):
        self.calls.append(agent)
        return self.script[agent].pop(0), Usage(agent, "fake", 10, 10, 0.0)

    def json(self, agent, system, messages, schema):
        return self._next(agent)

    def text(self, agent, system, messages):
        return self._next(agent)


class FakeRetriever:
    def search(self, query, k=5):
        chunk = {"entry": "Acne", "section": "Causes", "book_page": "25", "pdf_page": 40, "text":
"Sebum clogs pores."}
        return [Hit(chunk, 1.0, 0.9)]


def plan(intent="medical_question", reply=""):
    return Plan(intent=intent, language="English", search_query="acne causes" if not reply else "",
                direct_reply=reply, reasoning="test")


def test_accept_path():
    llm = FakeLLM({"planner": [plan()], "rag": ["Sebum clogs pores [1]."], "evaluator": [_ev()]})
    r = DoctorAgentSystem(llm, FakeRetriever()).ask("What causes acne?", Memory())
    assert r.answer == "Sebum clogs pores [1]." and not r.escalated
    assert r.confidence >= ACCEPT_THRESHOLD and r.attempts == 1
    assert llm.calls == ["planner", "rag", "evaluator"]


def test_revised_answer_replaces_original():
    ev = _ev(verdict="revise", revised_answer="Corrected [1].", unsupported_claims=["made up"])
```

```
    llm = FakeLLM({"planner": [plan()], "rag": ["Wrong claim."], "evaluator": [ev]})
    r = DoctorAgentSystem(llm, FakeRetriever()).ask("q", Memory())
    assert r.answer == "Corrected [1]." and r.revised


def test_requery_then_escalate():
    bad = _ev(verdict="insufficient_context", completeness=0.1, suggested_query="better query")
    llm = FakeLLM({"planner": [plan()], "rag": ["Not in book.", "Still not."], "evaluator": [bad,
bad]})
    r = DoctorAgentSystem(llm, FakeRetriever()).ask("Treatment for malaria?", Memory())
    assert r.escalated and r.attempts == MAX_ATTEMPTS
    assert llm.calls == ["planner", "rag", "evaluator", "rag", "evaluator"]
    assert "consult" in r.answer.lower()


def test_greeting_uses_no_retrieval():
    llm = FakeLLM({"planner": [plan("greeting", reply="Hi! Ask me a medical question.")]})
    r = DoctorAgentSystem(llm, FakeRetriever()).ask("hello", Memory())
    assert r.answer.startswith("Hi") and llm.calls == ["planner"]


def test_summarizer_runs_every_n_turns():
    n = 3
    llm = FakeLLM({"planner": [plan("greeting", reply="hi")] * n, "summarizer": ["summary v1"]})
    system, mem = DoctorAgentSystem(llm, FakeRetriever()), Memory()
    for _ in range(n):
        system.ask("hello", mem)
    assert mem.summary == "summary v1" and mem.turns_since_summary == 0
    assert llm.calls.count("summarizer") == 1


# ---------------------------------------------------------------- real retrieval (needs built index)
@pytest.mark.skipif(not (INDEX_DIR / "chunks.faiss").exists(), reason="index not built")
@pytest.mark.parametrize("query,entry", [
    ("What causes acne?", "Acne"),
    ("treatment for bronchitis", "Bronchitis"),
    ("side effects of acetaminophen", "Acetaminophen"),
    ("Bell's palsy prognosis", "Bell’s palsy"),  # regression: alias "ALS" used to match inside "palsy"
])
def test_retrieval_finds_right_entry(query, entry):
    from doctor_agent.retriever import HybridRetriever
    r = _retriever_cache.setdefault("r", HybridRetriever.load())
    assert entry in [h.chunk["entry"] for h in r.search(query, 5)]


_retriever_cache: dict = {}
```

```
pytest -q tests          # harus: 20 passed
```

Langkah 13 — Evaluasi (opsional)

⏱ ±15 menit mengetik + ±7 menit berjalan · biaya ±$1,2

Salin `eval/questions.json` dari `production` (27 pertanyaan dalam 7 kategori), tidak perlu diketik ulang:

```
cp ../production/eval/questions.json eval/
```

Kategori: in-book (EN/ID), bahasa awam, di luar buku (dengue, COVID-19, Zika, semaglutide, yang sudah dipastikan tidak ada di buku), non-medis, darurat, sapaan.

`run_eval.py` mengukur:

Metrik	Cara hitung

| intent accuracy | intent Planner = intent yang diharapkan | | Hit@5 / MRR | entri yang diharapkan ada di 5 sumber teratas / di peringkat berapa | | faithfulness, completeness, relevance, tone | rata-rata skor Evaluator pada jawaban yang diberikan | | in-book answer rate / out-of-book escalation rate | aspek keamanan | | citation validity | semua `[n]` merujuk ke sumber yang ada | | latensi & biaya | per pertanyaan | | **ablation** (gratis) | dense vs BM25 vs hybrid × pertanyaan mentah vs hasil rewrite Planner |

File: `eval/run_eval.py`

```
"""Evaluation of the whole multi-agent system on eval/questions.json.

Measures
  * routing      - Planner intent accuracy
  * retrieval    - Hit@5 and MRR of the expected Medical Book entry
  * answers      - Evaluator scores (faithfulness, completeness, relevance, tone), confidence,
                   revision / re-query rate, citation validity
  * safety       - in-book questions answered vs. out-of-book questions correctly escalated
  * cost/latency - USD and seconds per question
plus a free retrieval ablation: dense vs. BM25 vs. hybrid, raw question vs. Planner-rewritten query.

Run:  python eval/run_eval.py                (full run, uses the API, ~$1.5)
      python eval/run_eval.py --ablation-only (re-uses saved results, free)
"""
from __future__ import annotations

import argparse
import json
import re
import statistics as stats
import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

from doctor_agent.memory import Memory  # noqa: E402
from doctor_agent.orchestrator import DoctorAgentSystem  # noqa: E402

QUESTIONS = Path(__file__).parent / "questions.json"
OUT = Path(__file__).parent / "results"
MEDICAL = {"medical_question", "follow_up"}


def rank_of_expected(entries: list[str], expected: list[str]) -> int | None:
```

```
    for r, e in enumerate(entries, 1):
        if e in expected:
            return r
    return None


def citations_valid(answer: str, n_sources: int) -> bool:
    nums = [int(n) for n in re.findall(r"\[(\d+)\]", answer)]
    return all(1 <= n <= n_sources for n in nums)


def run_full(system: DoctorAgentSystem, questions: list[dict]) -> list[dict]:
    rows = []
    for i, q in enumerate(questions, 1):
        t0 = time.perf_counter()
        r = system.ask(q["question"], Memory())
        expected_intent = q.get("expected_intent", "medical_question")
        intent_ok = (r.intent in MEDICAL) if expected_intent == "medical_question" else (r.intent ==
expected_intent)
        entries = [h.chunk["entry"] for h in r.sources]
        rank = rank_of_expected(entries, q["expected_entries"]) if q["expected_entries"] else None
        ev = r.evaluation
        if q["category"].startswith("in_book"):
            outcome_ok = not r.escalated
        elif q["category"] == "out_of_book":
            outcome_ok = r.escalated
        else:
            outcome_ok = intent_ok
        row = {
            "id": q["id"], "category": q["category"], "lang": q["lang"], "question": q["question"],
            "intent": r.intent, "intent_ok": intent_ok,
            "search_query": r.plan.search_query if r.plan else "",
            "escalated": r.escalated, "outcome_ok": outcome_ok, "revised": r.revised, "attempts":
r.attempts,
            "confidence": r.confidence,
            "faithfulness": ev.faithfulness if ev else None, "completeness": ev.completeness if ev else
None,
            "relevance": ev.relevance if ev else None, "tone": ev.tone if ev else None,
            "verdict": ev.verdict if ev else None, "unsupported_claims": len(ev.unsupported_claims) if
ev else None,
            "hit_at_5": (rank is not None) if q["expected_entries"] else None,
            "reciprocal_rank": (1 / rank if rank else 0.0) if q["expected_entries"] else None,
            "citations_valid": citations_valid(r.answer, len(r.sources)) if r.sources else None,
            "sources": entries, "seconds": round(time.perf_counter() - t0, 2),
            "cost": round(r.cost, 5), "tokens": sum(r.tokens), "answer": r.answer,
        }
        rows.append(row)
        print(f"[{i:2}/{len(questions)}] {q['id']:6} intent={r.intent:16} esc={r.escalated!s:5} "
              f"conf={r.confidence if r.confidence is not None else '-'!s:5} ok={outcome_ok!s:5} "
              f"{row['seconds']:5.1f}s ${row['cost']:.4f}", flush=True)
    return rows


def mean(xs):
    xs = [x for x in xs if x is not None]
    return round(stats.mean(xs), 3) if xs else None


def rate(xs):
    xs = [x for x in xs if x is not None]
    return round(sum(bool(x) for x in xs) / len(xs), 3) if xs else None


def summarize(rows: list[dict]) -> dict:
    inb = [r for r in rows if r["category"].startswith("in_book")]
    oob = [r for r in rows if r["category"] == "out_of_book"]
    answered = [r for r in inb if not r["escalated"]]
    secs = sorted(r["seconds"] for r in rows)
```

```
    return {
        "n_questions": len(rows),
        "intent_accuracy": rate(r["intent_ok"] for r in rows),
        "retrieval_hit_at_5": rate(r["hit_at_5"] for r in inb),
        "retrieval_mrr": mean(r["reciprocal_rank"] for r in inb),
        "in_book_answer_rate": rate(not r["escalated"] for r in inb),
        "out_of_book_escalation_rate": rate(r["escalated"] for r in oob),
        "overall_outcome_accuracy": rate(r["outcome_ok"] for r in rows),
        "faithfulness": mean(r["faithfulness"] for r in answered),
        "completeness": mean(r["completeness"] for r in answered),
        "relevance": mean(r["relevance"] for r in answered),
        "tone": mean(r["tone"] for r in answered),
        "confidence_in_book": mean(r["confidence"] for r in answered),
        "confidence_out_of_book": mean(r["confidence"] for r in oob),
        "revision_rate": rate(r["revised"] for r in inb),
        "requery_rate": rate(r["attempts"] > 1 for r in rows if r["attempts"]),
        "citation_validity": rate(r["citations_valid"] for r in rows),
        "avg_latency_s": round(stats.mean(secs), 2),
        "p90_latency_s": secs[int(0.9 * (len(secs) - 1))],
        "avg_cost_usd": round(stats.mean(r["cost"] for r in rows), 4),
        "avg_cost_medical_usd": round(stats.mean(r["cost"] for r in rows if r["attempts"]), 4),
        "total_cost_usd": round(sum(r["cost"] for r in rows), 4),
    }


def ablation(system: DoctorAgentSystem, rows: list[dict], questions: list[dict]) -> list[dict]:
    """Retrieval only (free): which retriever + query form finds the expected entry?"""
    rewritten = {r["id"]: r["search_query"] for r in rows}
    inb = [q for q in questions if q["category"].startswith("in_book")]
    table = []
    for source in ("raw question", "planner rewrite"):
        for mode in ("dense", "sparse", "hybrid"):
            hits, rr = [], []
            for q in inb:
                query = q["question"] if source == "raw question" else (rewritten.get(q["id"]) or
q["question"])
                entries = [h.chunk["entry"] for h in system.retriever.search(query, 5, mode=mode)]
                rank = rank_of_expected(entries, q["expected_entries"])
                hits.append(rank is not None)
                rr.append(1 / rank if rank else 0.0)
            table.append({"query": source, "retriever": mode,
                          "hit_at_5": round(sum(hits) / len(hits), 3), "mrr": round(stats.mean(rr),
3)})
    return table


def write_report(summary: dict, abl: list[dict], rows: list[dict]) -> None:
    lines = ["# Evaluation results", "", "| Metric | Value |", "|---|---|"]
    lines += [f"| {k} | {v} |" for k, v in summary.items()]
    lines += ["", "## Retrieval ablation (in-book questions, Hit@5 / MRR)", "",
              "| Query | Retriever | Hit@5 | MRR |", "|---|---|---|---|"]
    lines += [f"| {a['query']} | {a['retriever']} | {a['hit_at_5']} | {a['mrr']} |" for a in abl]
    lines += ["", "## Per question", "",
              "| id | category | intent | escalated | conf | F | C | hit@5 | s | $ |", "|---|---|---|--
-|---|---|---|---|---|---|"]
    for r in rows:
        lines.append(f"| {r['id']} | {r['category']} | {r['intent']} | {r['escalated']} |
{r['confidence']} | "
                     f"{r['faithfulness']} | {r['completeness']} | {r['hit_at_5']} | {r['seconds']} |
{r['cost']} |")
    (OUT / "report.md").write_text("\n".join(lines) + "\n")


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--ablation-only", action="store_true")
    ap.add_argument("--ids", nargs="*", help="only run these question ids")
    args = ap.parse_args()
```

```
    questions = json.loads(QUESTIONS.read_text())
    if args.ids:
        questions = [q for q in questions if q["id"] in args.ids]
    OUT.mkdir(exist_ok=True)
    system = DoctorAgentSystem()

    if args.ablation_only:
        rows = json.loads((OUT / "results.json").read_text())
    else:
        rows = run_full(system, questions)
        (OUT / "results.json").write_text(json.dumps(rows, indent=1, ensure_ascii=False))

    summary = summarize(rows)
    abl = ablation(system, rows, questions)
    (OUT / "summary.json").write_text(json.dumps({"summary": summary, "ablation": abl}, indent=1))
    write_report(summary, abl, rows)
    print(json.dumps(summary, indent=1))
    for a in abl:
        print(a)


if __name__ == "__main__":
    main()
```

```
python eval/run_eval.py                 # penuh, ±$1,2
python eval/run_eval.py --ablation-only  # hanya ablation retrieval, gratis (butuh hasil run penuh)
```

Hasil tersimpan di `eval/results/` (`report.md`, `summary.json`, `results.json`).

Troubleshooting

Error / gejala	Penyebab	Solusi

| `ModuleNotFoundError: No module named 'config'` | menjalankan file langsung dari dalam folder | jalankan dari root folder dengan `python -m doctor_agent.ingest` | | `ModuleNotFoundError: anthropic` (atau lainnya) | venv belum aktif | `source .venv/bin/activate` | | `AuthenticationError` | API key salah / ada spasi / tanda kutip aneh | cek `.env` : `ANTHROPIC_API_KEY=sk-ant-...` tanpa spasi | | `FileNotFoundError: chunks.json` | ingestion belum dijalankan | `python -m doctor_agent.ingest` | | Pencarian pertama sangat lama | model embedding sedang diunduh / index sedang dibangun | tunggu ±2 menit, hanya sekali | | `BadRequestError` soal `budget_tokens` / `thinking` | memakai pola API lama | jangan kirim `thinking` ; pakai `output_config={"effort": ...}` | | `ValidationError` dari Pydantic | schema berbeda dengan field yang dipakai | pastikan nama field di `Plan` / `Evaluation` sama persis | | `Address already in use` (port 8501) | Streamlit lain masih jalan | `streamlit run app.py --server.port 8502` atau hentikan yang lama | | Semua pertanyaan dieskalasi | retrieval salah / index lama | hapus folder `index/` lalu `python -m doctor_agent.retriever` | | Jawaban bahasa Inggris padahal tanya bahasa Indonesia | `language` dari Planner tidak diteruskan | cek `SYSTEM.format(language=language)` di `rag.py` |

Checklist akhir

- ☐ `python -m doctor_agent.ingest` → 3.328 chunks
- ☐ `python -m doctor_agent.retriever` → acne → Acne
- ☐ `python scripts/smoke_test.py` → 4 pertanyaan berhasil, follow-up dipahami
- ☐ `streamlit run app.py` → chat jalan, badge confidence, panel trace
- ☐ Pertanyaan di luar buku → eskalasi
- ☐ (opsional) `pytest -q tests` → 20 passed
- ☐ (opsional) `python eval/run_eval.py` → metrik keluar

Bisa menjelaskan dengan kata-kata sendiri:

- ☐ kenapa chunking mengikuti struktur font buku
- ☐ apa itu embedding, BM25, dan RRF, serta kenapa digabung
- ☐ kenapa Planner menulis ulang query ke bahasa Inggris (74% → 100%)
- ☐ bagaimana Evaluator mendeteksi halusinasi dan bagaimana confidence dihitung
- ☐ kapan sistem mencari ulang dan kapan eskalasi (tabel ambang 0,70 / 0,45)
- ☐ kenapa memakai kombinasi model Haiku + Opus (Haiku < 1% biaya)
- ☐ cerita bug "ALS di dalam palsy" yang ditemukan oleh evaluasi

Selamat membangun! 🚀